

Correction détaillée — Exercices sur les variables

Exercice 1 : Tri

Réarranger le code pour afficher x, y, z dans l'ordre croissant (le plus petit, puis le milieu, puis le plus grand), avec $x = 45, y = 12, z = 17$.

- **Idée.** La fonction `min` donne directement le plus petit, `max` le plus grand. Pour la valeur du milieu, on remarque que si l'on retire à la somme des trois nombres le plus petit *et* le plus grand, il ne reste que celui du milieu :

$$\text{milieu} = (x + y + z) - \min(x, y, z) - \max(x, y, z).$$

Il suffit donc de **réordonner les trois lignes print** dans l'ordre min, milieu, max :

```
1 x = 45
2 y = 12
3 z = 17
4 print(min(x, y, z))
5 print(x + y + z - min(x, y, z) - max(x, y, z))
6 print(max(x, y, z))
```

- **Vérification** avec les valeurs données : $\min = 12, \max = 45, \text{milieu} = (45 + 12 + 17) - 12 - 45 = 74 - 57 = 17$. La console affiche donc, dans l'ordre :

```
12
17
45
```

- **Remarque.** On n'a modifié *que* l'ordre des instructions, sans introduire de nouvelle variable : c'est tout l'intérêt de l'astuce « somme moins les deux extrêmes ».

Exercice 2 : Le rectangle

Écrire un programme qui demande les dimensions d'un rectangle, puis calcule et affiche son périmètre et son aire.

- **Formules.** Pour un rectangle de longueur L et de largeur ℓ :

$$\mathcal{P} = 2(L + \ell) \quad \text{et} \quad \mathcal{A} = L \times \ell.$$

```
1 longueur = float(input("Longueur du rectangle : "))
2 largeur = float(input("Largeur du rectangle : "))
3 perimetre = 2 * (longueur + largeur)
4 aire = longueur * largeur
5 print("Périmètre :", perimetre)
6 print("Aire :", aire)
```

- **Point de vigilance.** L'instruction `input` renvoie toujours une **chaîne de caractères**. Pour pouvoir calculer, il faut la convertir en nombre : on utilise `float` (et non `int`) afin d'autoriser des dimensions décimales, par exemple 2,5.

- **Exemple.** Pour $L = 5$ et $\ell = 3$: périmètre $= 2 \times (5 + 3) = 16$ et aire $= 5 \times 3 = 15$.

Exercice 3 : Le triangle rectangle

Écrire un programme qui demande les côtés de l'angle droit d'un triangle rectangle, puis calcule et affiche son hypoténuse, son périmètre et son aire.

► **Formules.** Si a et b sont les deux côtés de l'angle droit, le théorème de Pythagore donne l'hypoténuse h :

$$h^2 = a^2 + b^2 \implies h = \sqrt{a^2 + b^2}.$$

Le périmètre vaut $a + b + h$ et l'aire $\frac{a \times b}{2}$ (les deux côtés de l'angle droit jouent le rôle de base et de hauteur).

```
1 from math import sqrt
2 a = float(input("Premier côté de l'angle droit : "))
3 b = float(input("Deuxième côté de l'angle droit : "))
4 hypotenuse = sqrt(a**2 + b**2)
5 perimetre = a + b + hypotenuse
6 aire = a * b / 2
7 print("Hypoténuse :", hypotenuse)
8 print("Périmètre :", perimetre)
9 print("Aire :", aire)
```

► **Variante sans importation.** La racine carrée est aussi la puissance $\frac{1}{2}$. On peut donc éviter le `from math import sqrt` et écrire l'hypoténuse à l'aide des seules puissances, déjà connues :

```
1 hypotenuse = (a**2 + b**2)**0.5
```

► **Exemple.** Pour $a = 3$ et $b = 4$: $h = \sqrt{9 + 16} = \sqrt{25} = 5$, périmètre $= 3 + 4 + 5 = 12$, aire $= \frac{3 \times 4}{2} = 6$.

Exercice 4 : Les œufs

Le programme `print(n//6)` affiche le nombre de boîtes de 6 œufs pour transporter n œufs.

► **1. Sur quelles valeurs de n le programme est-il correct ?**

L'opérateur `//` donne le quotient entier : `n//6` compte le nombre de boîtes **entièrement remplies**. Or, dès qu'il reste des œufs, il faut une boîte supplémentaire (partiellement remplie) pour les transporter. Le programme est donc correct uniquement lorsqu'il n'y a **aucun reste**, c'est-à-dire lorsque n est un **multiple de 6** :

$$n \in \{0, 6, 12, 18, \dots\}.$$

Par exemple, pour $n = 14$: `14//6` vaut 2, alors qu'il faut 3 boîtes (2 pleines + 1 pour les 2 œufs restants).

► **2. Pourquoi ne peut-on pas remplacer `n//6` par `n//6 + 1` ?**

Cela ajouterait *toujours* une boîte, y compris quand il n'y a pas de reste. Le résultat deviendrait alors faux pour les multiples de 6 : pour $n = 12$, `12//6 + 1` vaut 3, alors que 2 boîtes suffisent. Aucune des deux écritures (`n//6` ou `n//6 + 1`) n'est donc correcte pour *toutes* les valeurs de n .

► **3. Une solution correcte.**

Il faut ajouter une boîte *seulement* s'il reste des œufs, c'est-à-dire si `n%6` (le reste) est non nul :

```
1 n = int(input("Combien d'œufs : "))
2 if n % 6 == 0:
3     print(n // 6)
4 else:
5     print(n // 6 + 1)
```

► **Solution élégante (au programme « division euclidienne »).** On peut obtenir directement l'arrondi *par excès* sans test, en ajoutant 5 avant de diviser :

```
1 print((n + 5) // 6)
```

En effet, pour $n = 12$: $(12 + 5) // 6 = 17 // 6 = 2$; pour $n = 13$: $18 // 6 = 3$. Les deux programmes donnent les mêmes résultats.

Exercice 5 : Conversion

Lire une durée en secondes et l'exprimer sous la forme jours / heures / minutes / secondes (en utilisant // et %).

► **Idée.** On connaît les conversions : 1 jour = 86 400 s (soit 24×3600), 1 heure = 3 600 s, 1 minute = 60 s. À chaque étape, le quotient // donne le nombre d'unités, et le reste % la durée qu'il reste encore à convertir.

```
1 duree = int(input("Durée en secondes : "))
2 jours = duree // 86400
3 reste = duree % 86400
4 heures = reste // 3600
5 reste = reste % 3600
6 minutes = reste // 60
7 secondes = reste % 60
8 print(jours, "j", heures, "h", minutes, "min", secondes, "s")
```

► Trace pour `duree = 100 000 s`.

Étape	quotient (//)	reste (%)
$100\,000 = ? \times 86\,400$	jours = 1	13 600
$13\,600 = ? \times 3\,600$	heures = 3	2 800
$2\,800 = ? \times 60$	minutes = 46	40
reste	secondes = 40	–

La console affiche 1 j 3 h 46 min 40 s. *Vérification* : $1 \times 86\,400 + 3 \times 3\,600 + 46 \times 60 + 40 = 100\,000$.

Exercice 6

Qu'affiche l'algorithme ci-dessous ? (les variables `a` et `b` contiennent au départ un nombre saisi par l'utilisateur)

```
1 a = input('a = ')
2 b = input('b = ')
3 print("Avant, a =", a, "et b =", b)
4 tmp = a
5 a = b
6 b = tmp
7 print("Après, a =", a, "et b =", b)
```

► **Ce que fait l'algorithme : il échange (permuté) les contenus de a et b .**
 Suivons l'exécution en supposant que l'utilisateur saisit $a = 3$ puis $b = 5$:

Instruction	a	b	tmp
après les deux input	"3"	"5"	—
tmp = a	"3"	"5"	"3"
a = b	"5"	"5"	"3"
b = tmp	"5"	"3"	"3"

La console affiche donc :

```
Avant, a = 3 et b = 5
Après, a = 5 et b = 3
```

► **Rôle de la variable tmp.** Elle conserve provisoirement l'ancienne valeur de a . Sans elle, en écrivant directement $a = b$ puis $b = a$, on perdrait la valeur de a : à la deuxième ligne, a contiendrait déjà l'ancien b , et l'on obtiendrait a et b tous deux égaux à l'ancien b .

► **Remarque.** input renvoie des **chaînes de caractères** : ici a et b contiennent du texte ("3", "5"), mais l'échange fonctionne quel que soit le type des données.

Exercice 7

a et b valent respectivement 55 et 89. On remplace a par $a + b$, puis b par $a - b$, enfin a par $a - b$.

► **1. Que contiennent a et b à la fin ?**

Suivons pas à pas, en gardant en tête qu'à chaque ligne le membre de droite est calculé *avant* d'être rangé dans la variable de gauche :

Instruction	a	b
valeurs initiales	55	89
$a = a + b$	$55 + 89 = 144$	89
$b = a - b$	144	$144 - 89 = 55$
$a = a - b$	$144 - 55 = 89$	55

À la fin : $a = 89$ et $b = 55$. Les contenus de a et b ont été **échangés**.

► **2. Programme Python.**

```
1 a = 55
2 b = 89
3 a = a + b
4 b = a - b
5 a = a - b
6 print("a =", a, "et b =", b)
```

► **Comparaison avec l'exercice 6.** On obtient le même échange, mais **sans variable intermédiaire** : on utilise ici uniquement des sommes et des différences. En contrepartie, cette méthode ne fonctionne qu'avec des *nombres*, tandis que la méthode avec **tmp** (exercice 6) fonctionne pour tout type de données.